

CHƯƠNG 03: GIẢI QUYẾT VẤN ĐỀ BẰNG TÌM KIẾM

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 3

- ◇ 3.1 Các tác nhân giải quyết vấn đề (Problem-Solving Agents)
- ◇ 3.2 Các bài toán ví dụ (Example Problems)
- ◇ 3.3 Thuật toán Tìm kiếm (Search Algorithms)
- ◇ 3.4 Chiến lược Tìm kiếm Không thông tin (Uninformed Search)
- ◇ 3.5 Tìm kiếm Có thông tin (Informed Heuristic Search)
- ◇ 3.6 Hàm Heuristic (Heuristic Functions)

3.1 Các tác nhân giải quyết vấn đề

◇ **Tác nhân giải quyết vấn đề** là dạng tác nhân dựa trên mục tiêu, suy nghĩ về tương lai để tìm ra chuỗi hành động dẫn tới trạng thái mục tiêu.

◇ Quy trình 4 bước của Tác nhân:

- **Định hình mục tiêu (Goal formulation):** Tập các trạng thái thể giới thỏa mãn mục tiêu.
- **Định hình bài toán (Problem formulation):** Xác định không gian trạng thái, hành động, và mô hình chuyển đổi.
- **Tìm kiếm (Search):** Mô phỏng các chuỗi hành động trong đầu để tìm ra chuỗi tối ưu (Solution).
- **Thực thi (Execute):** Lần lượt thực hiện các hành động trong chuỗi giải pháp.

Công thức bài toán (Problem Formulation)

◇ Một bài toán được định nghĩa qua 5 thành phần chính:

- **Trạng thái ban đầu (Initial state):** Trạng thái tác nhân bắt đầu (VD: Đang ở thành phố Arad).
- **Các hành động có thể (Actions):** Tập $\mathcal{A}(s)$ là các hành động hợp lệ từ trạng thái s .
- **Mô hình chuyển đổi (Transition model):** Hàm $T(s, a) = s'$, trả về trạng thái s' đạt được khi thực hiện a tại s .
- **Kiểm tra mục tiêu (Goal test):** Hàm kiểm tra xem trạng thái s có phải là mục tiêu không.
- **Chi phí đường đi (Path cost):** Hàm $c(s, a, s')$ tính chi phí chuyển từ s sang s' . Chi phí luôn dương.

Ví dụ bài toán tìm đường ở Romania

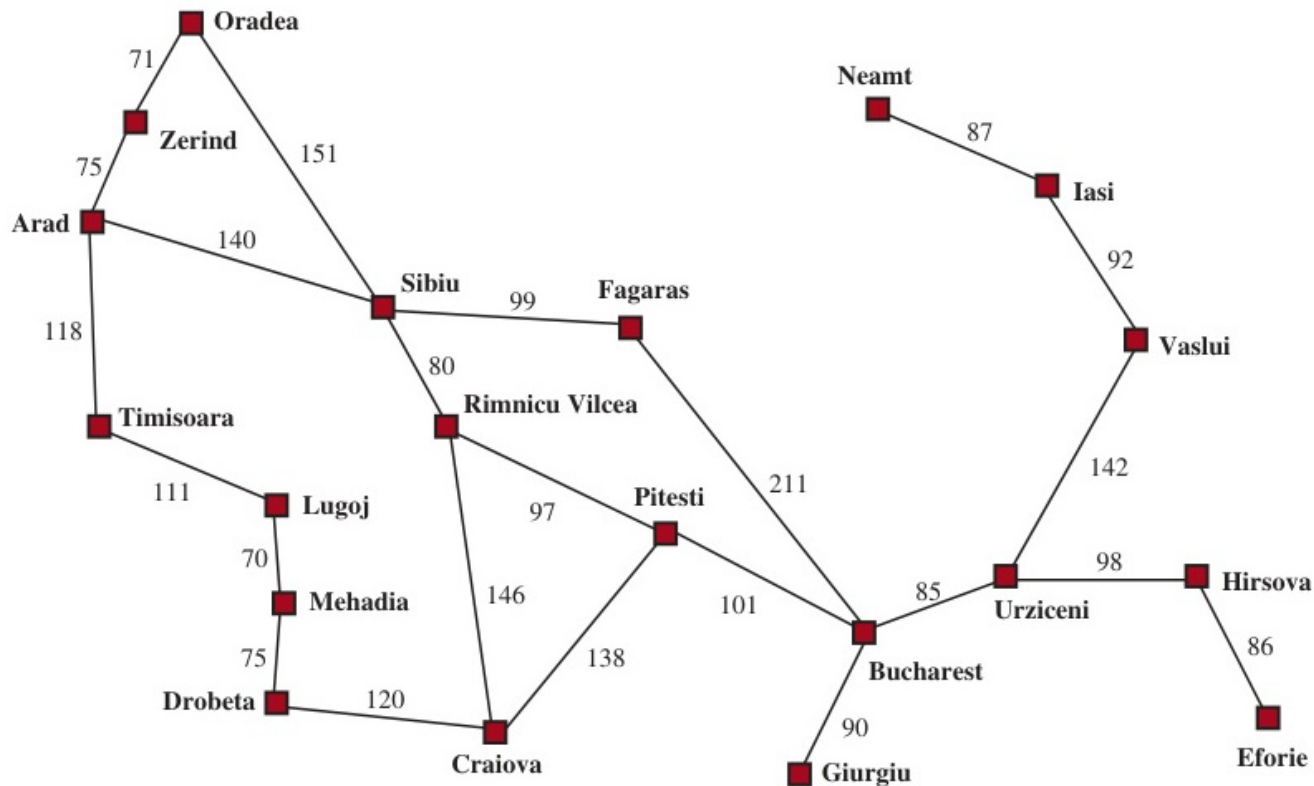


Figure 1: Bản đồ Romania với khoảng cách giữa các thành phố.

3.2 Các bài toán ví dụ (Toy Problems)

◇ Thế giới máy hút bụi (Vacuum-cleaner world):

- **Trạng thái:** Vị trí của robot (2 ô) $\times$ Trạng thái của ô (sạch/dơ).
Có $2 \times 2^2 = 8$ trạng thái.
- **Hành động:** Trái (Left), Phải (Right), Hút (Suck).
- **Goal test:** Tất cả các ô đều sạch.

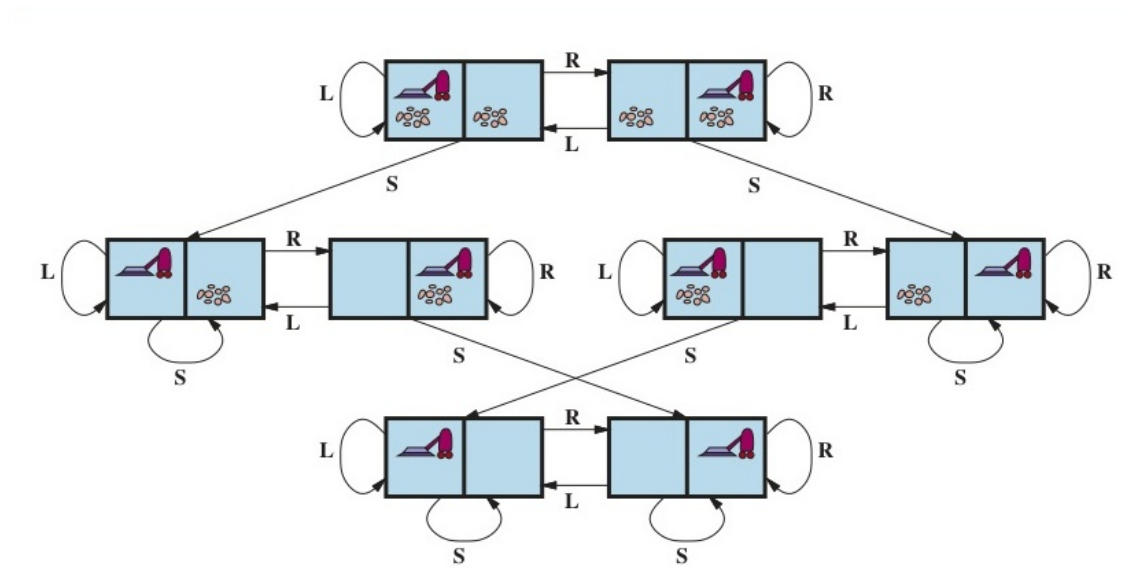


Figure 2: Không gian trạng thái của bài toán máy hút bụi với 8 trạng thái.

Các bài toán ví dụ (Toy Problems - tt)

◇ Bài toán 8-puzzle (Trượt ngói):

- Không gian trạng thái: $9!/2 = 181,440$ trạng thái hợp lệ.
- Hành động: Trượt ô trống (Lên, Xuống, Trái, Phải).

◇ Bài toán 8 Hậu (8-Queens):

- Đặt 8 quân hậu lên bàn cờ vua sao cho không quân nào an quân nào.
- State formulation: Có thể đặt từng quân hậu một (Tang dần) hoặc di chuyển quân hậu (Hoàn chỉnh).

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Figure 3: Bài toán 8-puzzle và Bài toán 8-Queens.

3.3 Thuật toán Tìm kiếm (Search Algorithms)

◇ Tìm kiếm là quá trình xây dựng một **Cây tìm kiếm (Search tree)** bao trùm Không gian trạng thái.

◇ Các thuật ngữ trên cây tìm kiếm:

- **Node (Nút):** Đại diện cho một trạng thái, nhưng chứa thêm thông tin (parent, action, path cost $g(n)$, depth).
- **Khai triển (Expand):** Áp dụng hành động để sinh ra các Node con từ Node hiện tại.
- **Biên (Frontier):** Tập hợp các node đã được sinh ra nhưng chưa được khai triển.
- **Tập đã duyệt (Explored set / Closed list):** Ghi nhớ các trạng thái đã xét để tránh vòng lặp trên đồ thị.

Ví dụ duyệt cây tìm kiếm

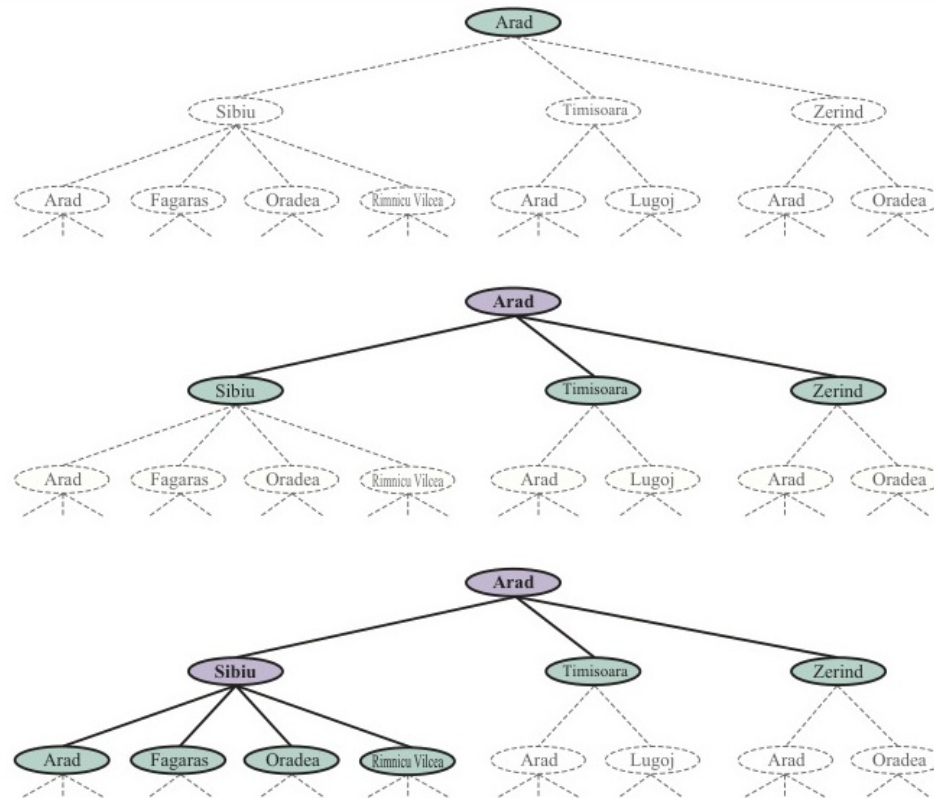


Figure 4: Quá trình mở rộng cây tìm kiếm từ Arad đi Bucharest.

Tìm kiếm Đồ thị (Graph Search)

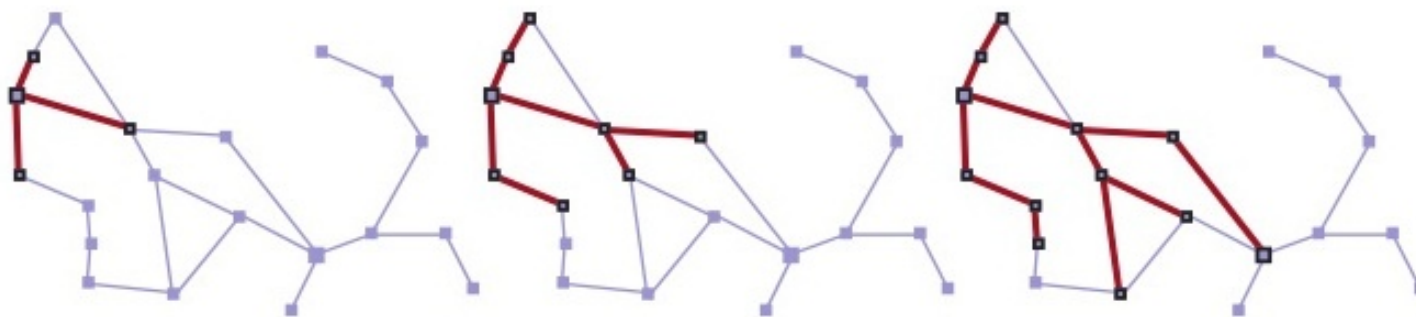


Figure 5: Sự phát triển của cây tìm kiếm đồ thị trên bản đồ Romania.

Sự phân tách trong Tìm kiếm Đồ thị

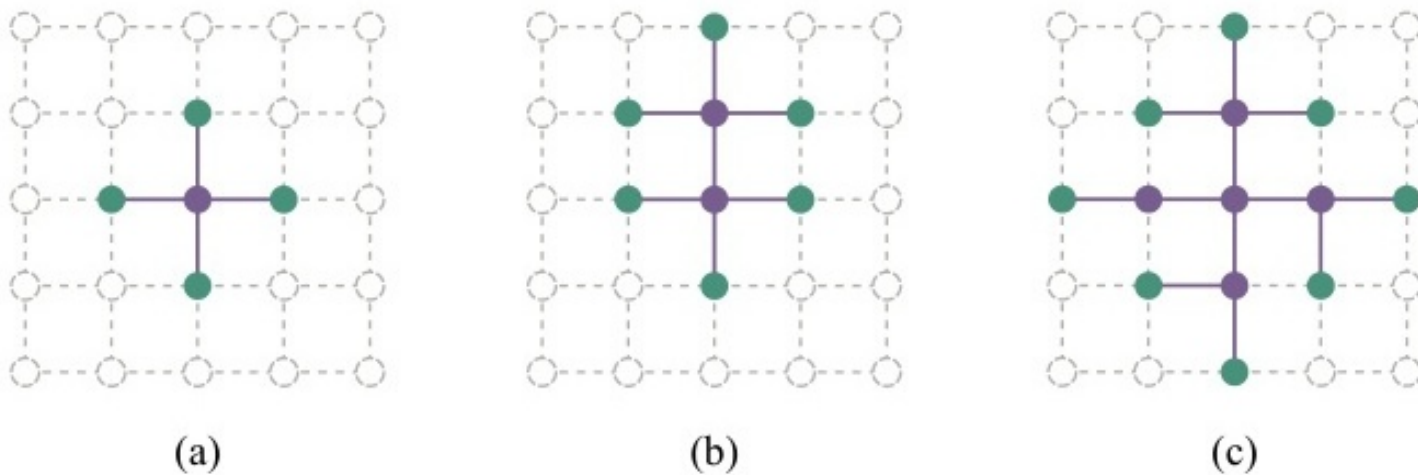


Figure 6: Thuộc tính tách biệt (Separation property) của tìm kiếm đồ thị.

Thuật toán Tìm kiếm Cây và Đồ thị

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure

function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

Figure 7: Thuật toán tổng quát cho Tìm kiếm dạng Cây và Đồ thị.

Đánh giá thuật toán Tìm kiếm

◇ Một thuật toán tìm kiếm được đánh giá theo 4 tiêu chí:

- **Tính hoàn thiện (Completeness):** Thuật toán có chắc chắn tìm ra nghiệm nếu nghiệm tồn tại?
- **Tính tối ưu (Optimality):** Thuật toán có tìm ra nghiệm có chi phí nhỏ nhất không?
- **Độ phức tạp thời gian (Time complexity):** Mất bao lâu để tìm ra nghiệm?
- **Độ phức tạp không gian (Space complexity):** Cần bao nhiêu bộ nhớ trong quá trình chạy?

◇ **Kí hiệu độ phức tạp:**

b : Hệ số rẽ nhánh (branching factor).

d : Độ sâu của nghiệm nông nhất (depth of shallowest goal).

m : Chiều dài tối đa của đường đi trên đồ thị.

3.4 Chiến lược Tìm kiếm Không thông tin

◇ **Tìm kiếm Không thông tin (Uninformed Search)** hay Tìm kiếm mù (Blind search) là chiến lược không sử dụng bất kỳ thông tin nào khác ngoài cấu trúc cơ bản của bài toán (Không biết hướng nào đến mục tiêu gần hơn).

◇ Các thuật toán phổ biến:

- Tìm kiếm theo chiều rộng (Breadth-first search - BFS).
- Tìm kiếm chi phí đồng nhất (Uniform-cost search - UCS / Dijkstra).
- Tìm kiếm theo chiều sâu (Depth-first search - DFS).
- Tìm kiếm sâu dần (Depth-limited search & Iterative deepening - IDS).

Tìm kiếm theo Chiều rộng (BFS)

- ◇ BFS khai triển các node ở độ sâu d trước khi khai triển node ở $d + 1$.
- ◇ **Cấu trúc Frontier:** Hàng đợi (Queue - FIFO).
- ◇ **Đánh giá:**
 - **Complete:** Có (nếu b hữu hạn).
 - **Optimal:** Có (nếu mọi chi phí bước đều bằng nhau).
 - **Time:** $\mathcal{O}(b^d)$.
 - **Space:** $\mathcal{O}(b^d)$ (Frontier giữ toàn bộ các node ở tầng dưới cùng). Đây là nhược điểm cực kì lớn khiến BFS ít được dùng trong thực tế!

Cây tìm kiếm Chiều rộng (BFS)

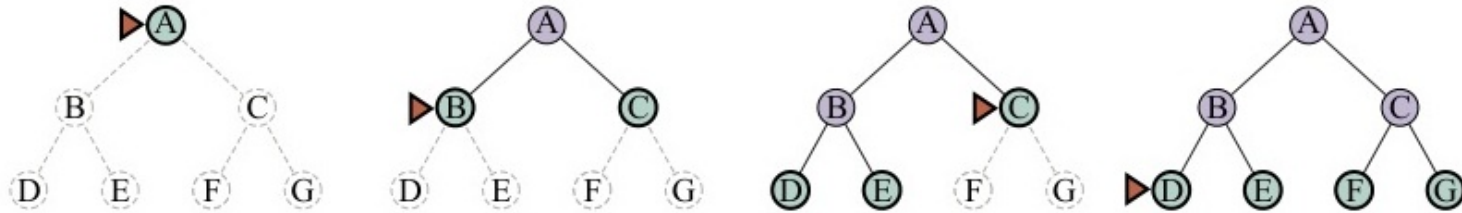


Figure 8: Quá trình mở rộng nút trong BFS trên cây nhị phân.

Thuật toán Tìm kiếm theo Chiều rộng

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node  $\leftarrow$  NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier  $\leftarrow$  a FIFO queue, with node as an element
  reached  $\leftarrow$  {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node  $\leftarrow$  POP(frontier)
    for each child in EXPAND(problem, node) do
      s  $\leftarrow$  child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure

function UNIFORM-COST-SEARCH(problem) returns a solution node, or failure
  return BEST-FIRST-SEARCH(problem, PATH-COST)
```

Figure 9: Cấu trúc thuật toán Breadth-First Search.

Tìm kiếm Chi phí đồng nhất (UCS)

- ◇ Khai triển node có chi phí đường đi từ đầu $g(n)$ nhỏ nhất, thay vì node nông nhất.
- ◇ **Cấu trúc Frontier:** Hàng đợi ưu tiên (Priority Queue).
- ◇ **Đánh giá:**
 - **Complete:** Có (nếu b hữu hạn và mọi chi phí bước $\geq \epsilon > 0$).
 - **Optimal:** Có. Luôn đảm bảo tìm ra đường đi có tổng chi phí tối ưu nhất.
 - **Time & Space:** $\mathcal{O}(b^{1+\lceil C^*/\epsilon \rceil})$, với C^* là chi phí của nghiệm tối ưu.

Ví dụ Tìm kiếm Chi phí đồng nhất

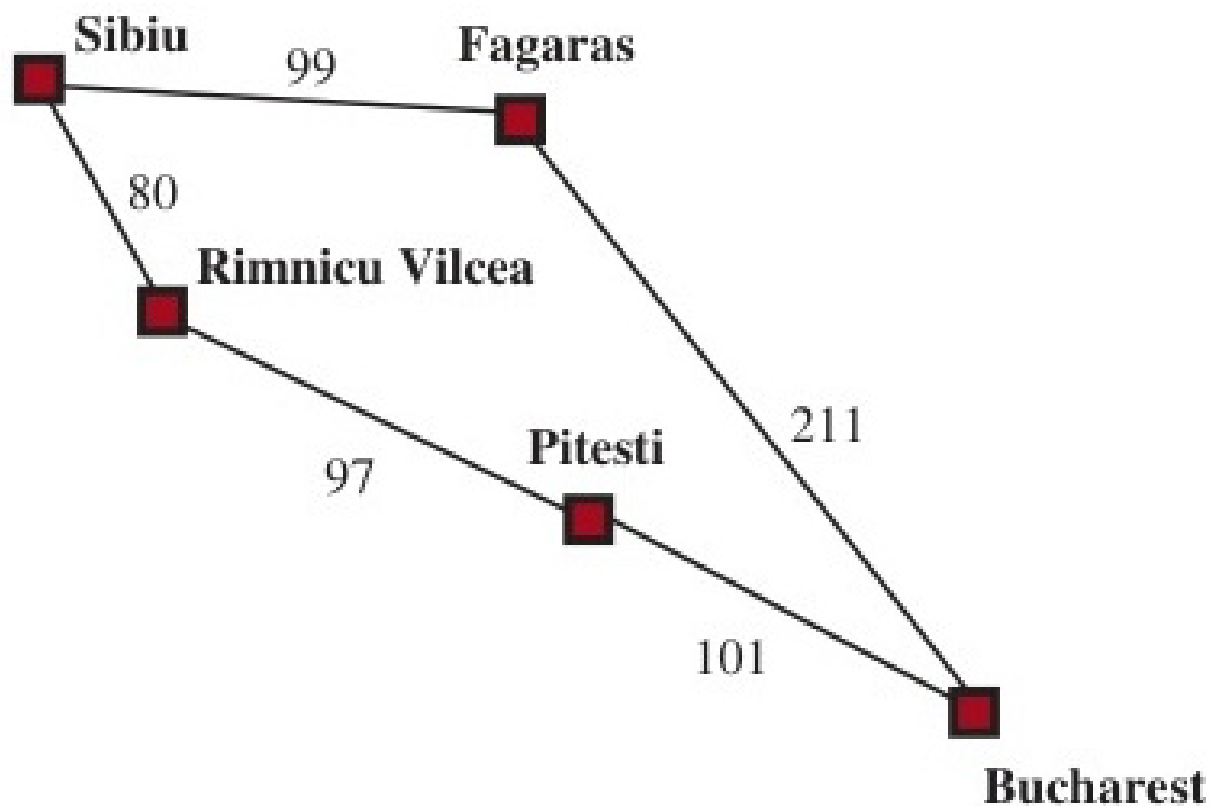


Figure 10: Một phần không gian trạng thái ở Romania để minh họa thuật toán UCS.

Cây tìm kiếm Chi phí đồng nhất (UCS)

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure
  for depth = 0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result

function DEPTH-LIMITED-SEARCH(problem,  $\ell$ ) returns a node or failure or cutoff
  frontier  $\leftarrow$  a LIFO queue (stack) with NODE(problem.INITIAL) as an element
  result  $\leftarrow$  failure
  while not IS-EMPTY(frontier) do
    node  $\leftarrow$  POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    if DEPTH(node) >  $\ell$  then
      result  $\leftarrow$  cutoff
    else if not IS-CYCLE(node) do
      for each child in EXPAND(problem, node) do
        add child to frontier
  return result
```

Figure 11: Mở rộng đường đi dựa trên tổng chi phí $g(n)$ trong thuật toán UCS.

Tìm kiếm theo Chiều sâu (DFS)

- ◇ Luôn khai triển node sâu nhất trong Frontier chưa được khám phá.
- ◇ **Cấu trúc Frontier:** Ngăn xếp (Stack - LIFO).
- ◇ **Đánh giá:**
 - **Complete:** Chỉ hoàn thiện trên đồ thị hữu hạn (cần lưu Explored set). Thất bại ở không gian vô hạn.
 - **Optimal:** Không. Tìm thấy bất kỳ nghiệm nào là dừng.
 - **Time:** $\mathcal{O}(b^m)$ (Có thể cực kì tồi tệ nếu $m \gg d$).
 - **Space:** $\mathcal{O}(bm)$ (Cực kỳ tiết kiệm bộ nhớ, chỉ lưu trữ 1 đường đi từ gốc tới lá hiện tại!).

Cây tìm kiếm Chiều sâu (DFS)

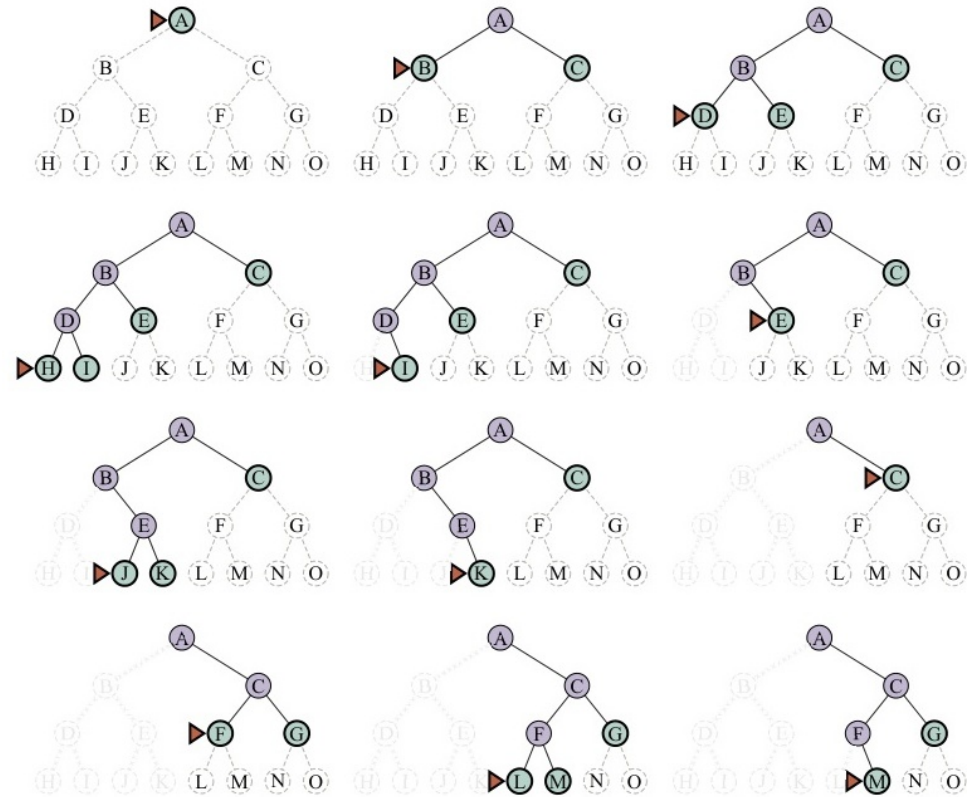


Figure 12: Quá trình tìm kiếm theo chiều sâu đầu tiên trên cây nhị phân.

Tìm kiếm Sâu lặp dần (Iterative Deepening - IDS)

- ◇ **Chiến lược:** Kết hợp ưu điểm không gian nhỏ của DFS và tính hoàn thiện của BFS.
- ◇ **Cách làm:** Chạy DFS với độ sâu giới hạn $limit = 0$, nếu không thấy thì chạy lại với $limit = 1, limit = 2, \dots$ cho đến khi tìm được nghiệm.
- ◇ **Đánh giá:**
 - **Complete:** Có.
 - **Optimal:** Có (nếu chi phí bước bằng nhau).
 - **Time:** $\mathcal{O}(b^d)$.
 - **Space:** $\mathcal{O}(bd)$ (Bộ nhớ siêu tiết kiệm tương tự DFS).
- ◇ **Khuyến dùng:** IDS là thuật toán không thông tin mặc định nên chọn khi không gian tìm kiếm lớn và độ sâu nghiệm chưa biết.

Thuật toán Tìm kiếm Sâu giới hạn

```

function BIBF-SEARCH( $problem_F, f_F, problem_B, f_B$ ) returns a solution node, or failure
   $node_F \leftarrow \text{NODE}(problem_F.INITIAL)$  // Node for a start state
   $node_B \leftarrow \text{NODE}(problem_B.INITIAL)$  // Node for a goal state
   $frontier_F \leftarrow$  a priority queue ordered by  $f_F$ , with  $node_F$  as an element
   $frontier_B \leftarrow$  a priority queue ordered by  $f_B$ , with  $node_B$  as an element
   $reached_F \leftarrow$  a lookup table, with one key  $node_F.STATE$  and value  $node_F$ 
   $reached_B \leftarrow$  a lookup table, with one key  $node_B.STATE$  and value  $node_B$ 
   $solution \leftarrow \text{failure}$ 
  while not TERMINATED( $solution, frontier_F, frontier_B$ ) do
    if  $f_F(\text{TOP}(frontier_F)) < f_B(\text{TOP}(frontier_B))$  then
       $solution \leftarrow \text{PROCEED}(F, problem_F, frontier_F, reached_F, reached_B, solution)$ 
    else  $solution \leftarrow \text{PROCEED}(B, problem_B, frontier_B, reached_B, reached_F, solution)$ 
  return  $solution$ 

function PROCEED( $dir, problem, frontier, reached, reached_2, solution$ ) returns a solution
  // Expand node on frontier; check against the other frontier in  $reached_2$ .
  // The variable "dir" is the direction: either F for forward or B for backward.
   $node \leftarrow \text{POP}(frontier)$ 
  for each  $child$  in EXPAND( $problem, node$ ) do
     $s \leftarrow child.STATE$ 
    if  $s$  not in  $reached$  or  $\text{PATH-COST}(child) < \text{PATH-COST}(reached[s])$  then
       $reached[s] \leftarrow child$ 
      add  $child$  to  $frontier$ 
    if  $s$  is in  $reached_2$  then
       $solution_2 \leftarrow \text{JOIN-NODES}(dir, child, reached_2[s])$ 
      if  $\text{PATH-COST}(solution_2) < \text{PATH-COST}(solution)$  then
         $solution \leftarrow solution_2$ 
  return  $solution$ 

```

Figure 13: Thuật toán Depth-Limited Search.

Thuật toán Tìm kiếm Sâu lặp dần (IDS)

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Figure 14: Thuật toán Iterative Deepening Search.

Minh họa Tìm kiếm Sâu lặp dần (IDS)

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 15: Sự gia tăng giới hạn độ sâu (Limit) lặp đi lặp lại trong thuật toán IDS.

Minh họa chi tiết Tìm kiếm Sâu lặp dần

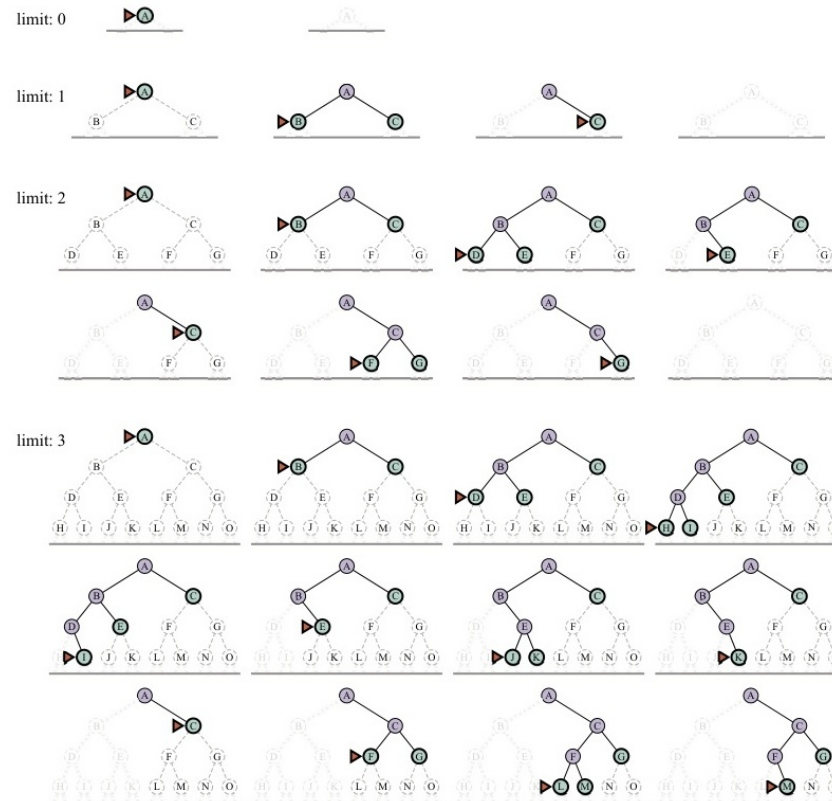


Figure 16: Bốn lần lặp của thuật toán IDS trên cây nhị phân.

3.5 Tìm kiếm Có thông tin (Heuristic Search)

◇ Thay vì tìm kiếm mù quáng, thuật toán có thông tin sử dụng kiến thức đặc thù về vấn đề (Problem-specific knowledge) để rút ngắn thời gian.

◇ **Hàm Heuristic $h(n)$** : Ước lượng chi phí đường đi rẻ nhất từ node n tới mục tiêu.

- Đặc điểm: Nếu n là đích thì $h(n) = 0$.
- Ví dụ bài toán tìm đường ở Romania: Hàm $h_{SLD}(n)$ là Khoảng cách đường chim bay (Straight-line distance) từ thành phố n đến đích (Bucharest).

◇ Khai triển node dựa trên **Hàm đánh giá $f(n)$** . Frontier là Priority Queue được sắp xếp tối thiểu theo $f(n)$.

Tìm kiếm Tham lam (Greedy Best-First Search)

- ◇ **Chiến lược:** Khai triển node trông có vẻ "gần" mục tiêu nhất.
- ◇ **Hàm đánh giá:** $f(n) = h(n)$.
- ◇ **Đánh giá:**
 - Thuật toán thường đi thẳng tới mục tiêu rất nhanh, tiết kiệm thời gian và bộ nhớ cực kì hiệu quả ở những bài toán thực tế.
 - Tuy nhiên: **KHÔNG** hoàn thiện và **KHÔNG** tối ưu. Tác nhân có thể rơi vào đường cụt hoặc vòng lặp.
 - Cây tìm kiếm của Tham lam bị thu hút quá mức vào heuristic mà bỏ qua việc tiết kiệm tổng khoảng cách đã đi.

Đồ thị tìm kiếm Tham lam

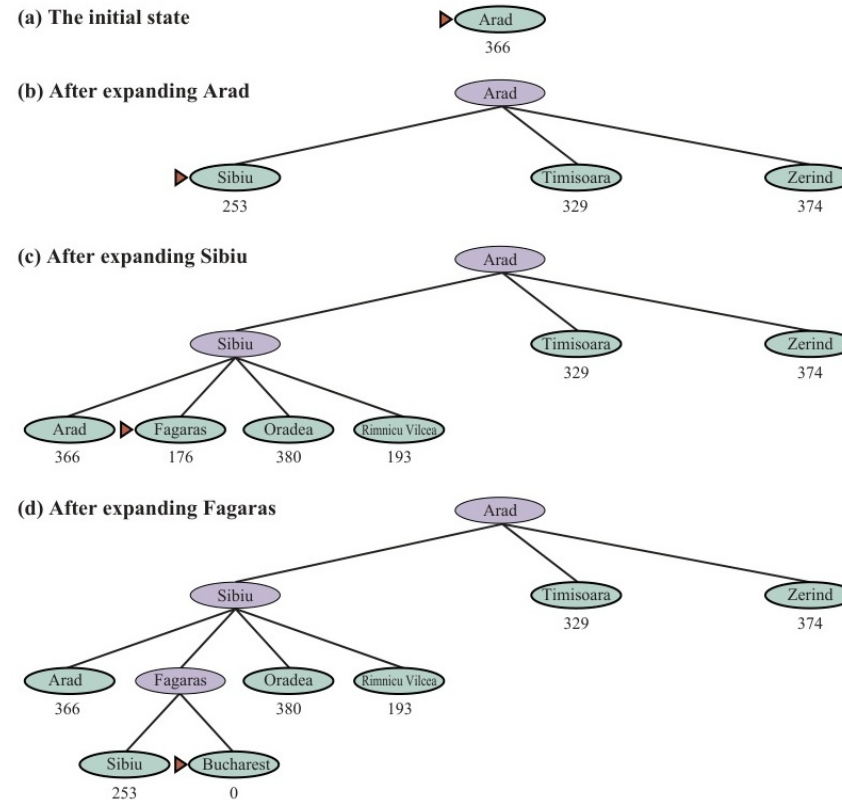


Figure 17: Quá trình tìm đường bằng thuật toán Greedy Best-First Search.

Tìm kiếm A* (A-Star Search)

◇ **Chiến lược:** Kết hợp chi phí đã đi và chi phí ước lượng còn lại.

◇ **Hàm đánh giá tổng:** $f(n) = g(n) + h(n)$

- $g(n)$: Chi phí đường đi thực tế từ điểm gốc tới n .
- $h(n)$: Chi phí ước lượng rẻ nhất từ n tới mục tiêu.
- $f(n)$: Tổng chi phí ước lượng của toàn bộ đường đi qua n .

◇ **Tầm quan trọng của A*:** Đây là thuật toán tìm kiếm "quốc dân" (Best-known algorithm), đạt được cả tính hoàn thiện (Completeness) và tính tối ưu tuyệt đối (Optimality) nếu hàm $h(n)$ thỏa mãn một số điều kiện nhất định.

Đồ thị tìm kiếm A*

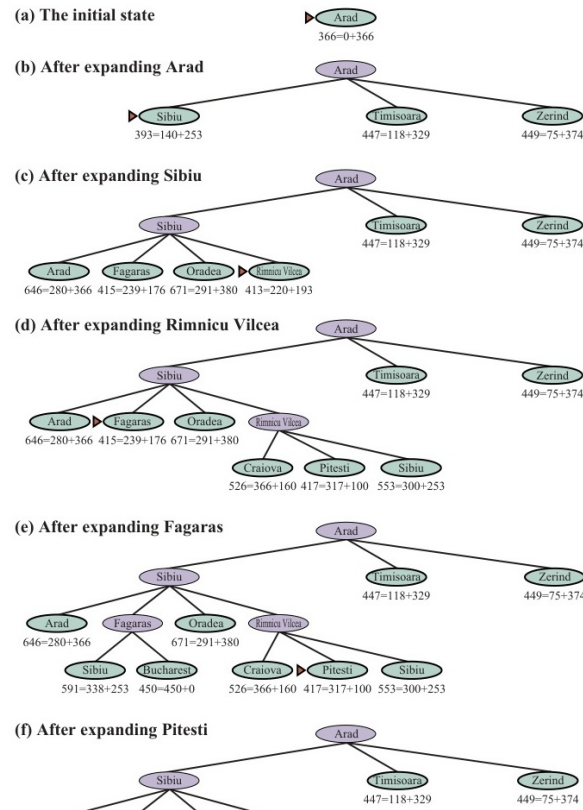


Figure 18: Quá trình mở rộng bằng A* Search, cân bằng giữa $g(n)$ và $h(n)$.

3.6 Điều kiện của Hàm Heuristic

◇ **1. Chấp nhận được (Admissible):** Cần thiết để A^* tối ưu trên Tìm kiếm Nút (Tree Search).

$$0 \leq h(n) \leq h^*(n)$$

Trong đó $h^*(n)$ là chi phí thực sự rẻ nhất để đến đích. Heuristic không bao giờ được "đánh giá quá cao" (overestimate) độ khó thực tế của phần đường còn lại.

◇ **2. Nhất quán (Consistent / Monotonic):** Cần thiết để A^* tối ưu trên Tìm kiếm Đồ thị (Graph Search).

$$h(n) \leq c(n, a, n') + h(n')$$

Tức là ước lượng $h(n)$ luôn thỏa mãn bất đẳng thức tam giác so với thực tế chi phí chuyển tiếp bước $c(n, a, n')$. Mọi hàm consistent đều là admissible.

Tính Nhất quán của Hàm Heuristic

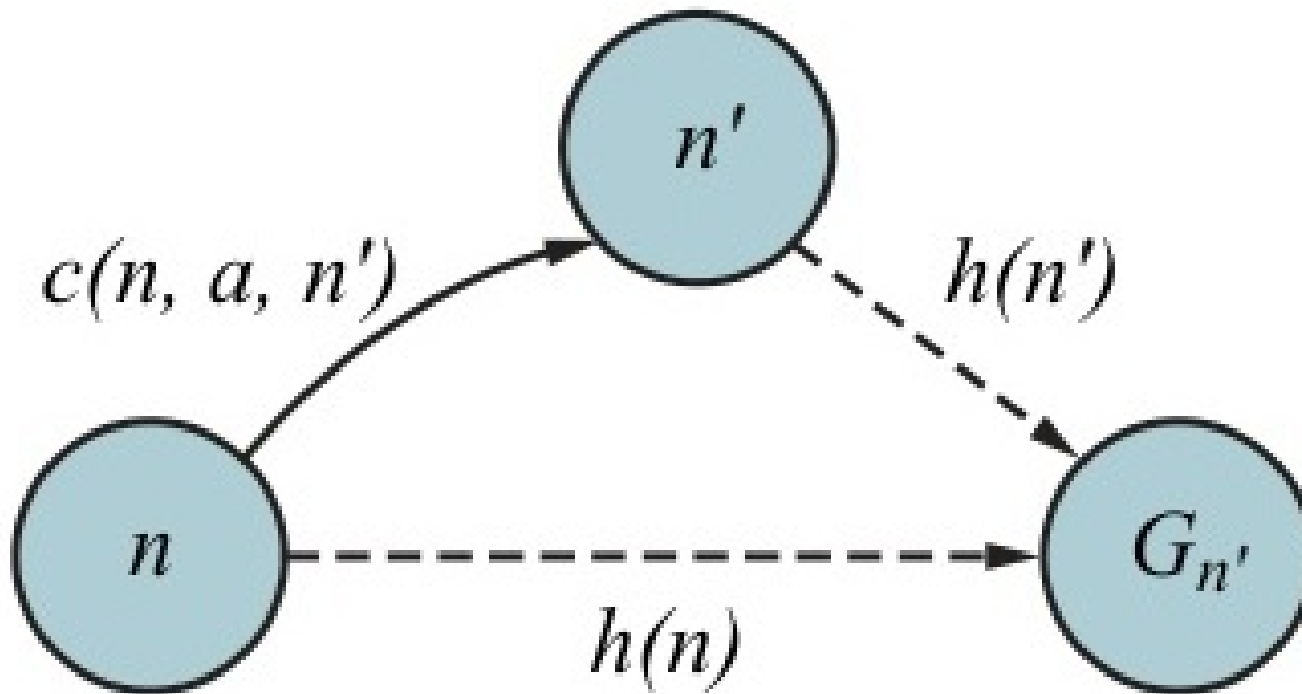


Figure 19: Bất đẳng thức tam giác minh họa tính nhất quán (Consistency).

Các đường đồng mức trong A*

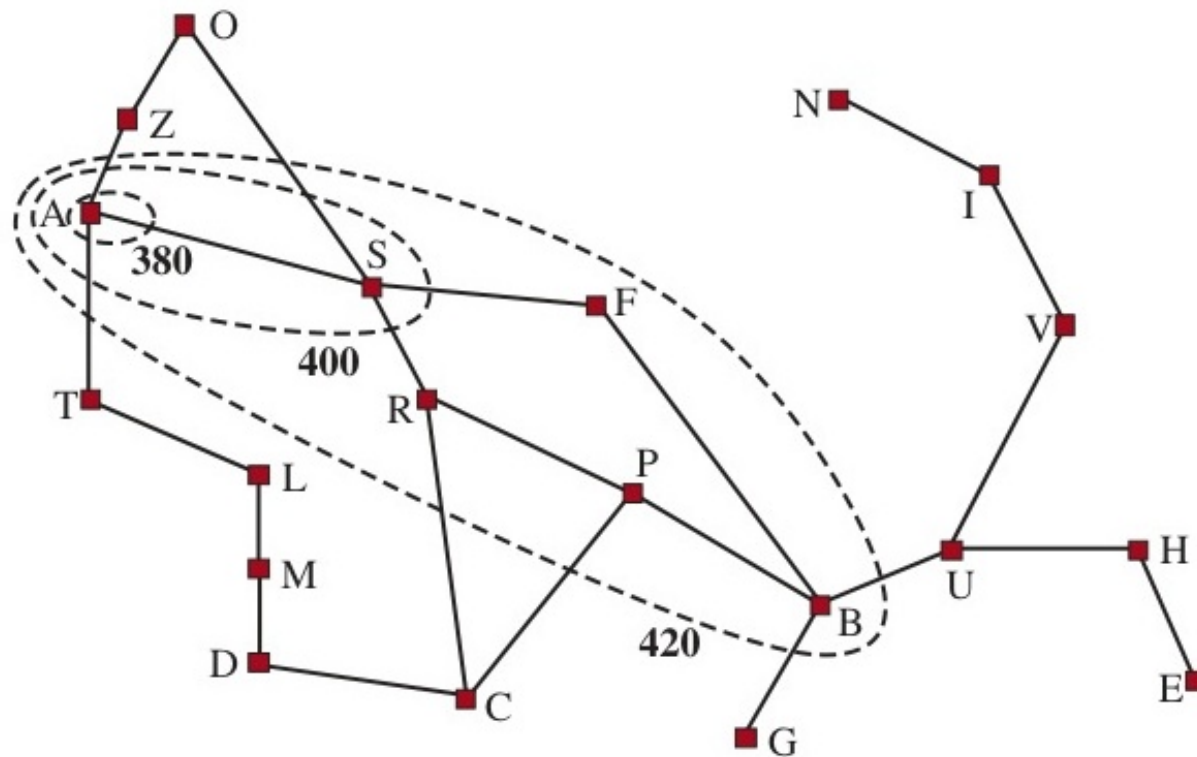


Figure 20: Các đường đồng mức $f(n)$ trên bản đồ Romania.

Sự Vượt trội của Heuristic (Dominance)

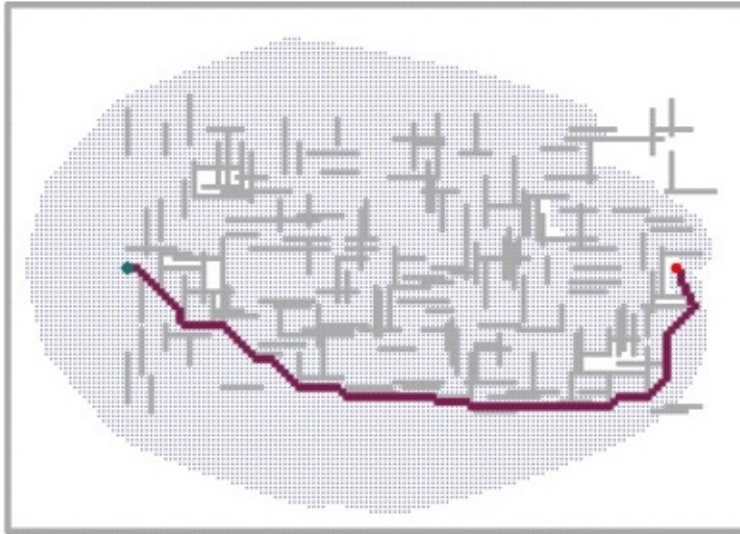
- ◇ Một hàm heuristic tốt sẽ làm giảm đáng kể số lượng node cần mở (Hệ số rẽ nhánh hiệu dụng - Effective branching factor b^* gần với 1).
- ◇ Giả sử có hai hàm $h_1(n)$ và $h_2(n)$ đều admissible. Nếu:

$$\forall n, h_2(n) \geq h_1(n)$$

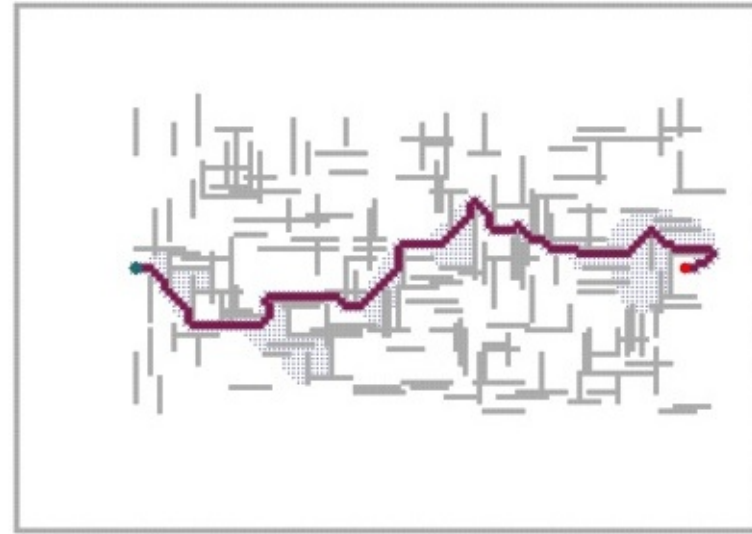
Thì ta nói h_2 **vượt trội (dominates)** h_1 .

- ◇ Tính chất: Sử dụng hàm h_2 vượt trội cho A^* sẽ luôn mở rộng số lượng node bằng hoặc ít hơn (tốt hơn về tốc độ) so với khi dùng hàm h_1 .
- ◇ Việc thiết kế hàm Heuristic vượt trội thường bằng cách giải chính xác một **Bài toán nới lỏng (Relaxed problem)**.

Tìm kiếm A^* có trọng số (Weighted A^*)



(a)



(b)

Figure 21: So sánh giữa A^* tiêu chuẩn và A^* có trọng số.

Tìm kiếm tốt nhất đầu tiên đệ quy (RBFS)

```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution or failure  
    solution, fvalue  $\leftarrow$  RBFS(problem, NODE(problem.INITIAL),  $\infty$ )  
    return solution  
  
function RBFS(problem, node, f_limit) returns a solution or failure, and a new f-cost limit  
    if problem.IS-GOAL(node.STATE) then return node  
    successors  $\leftarrow$  LIST(EXPAND(node))  
    if successors is empty then return failure,  $\infty$   
    for each s in successors do           // update f with value from previous search  
        s.f  $\leftarrow$  max(s.PATH-COST + h(s), node.f)  
    while true do  
        best  $\leftarrow$  the node in successors with lowest f-value  
        if best.f > f_limit then return failure, best.f  
        alternative  $\leftarrow$  the second-lowest f-value among successors  
        result, best.f  $\leftarrow$  RBFS(problem, best, min(f_limit, alternative))  
        if result  $\neq$  failure then return result, best.f
```

Figure 22: Thuật toán Recursive Best-First Search.

Các giai đoạn của RBFS

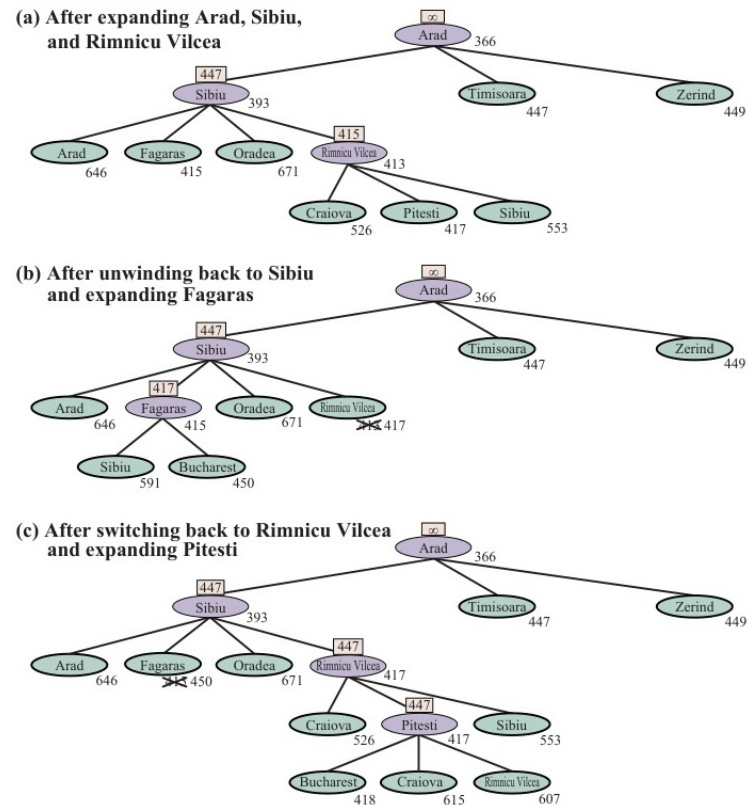


Figure 23: Các giai đoạn mở rộng nút của thuật toán RBFS trên bản đồ Romania.

Tìm kiếm Hai chiều (Bidirectional Search)

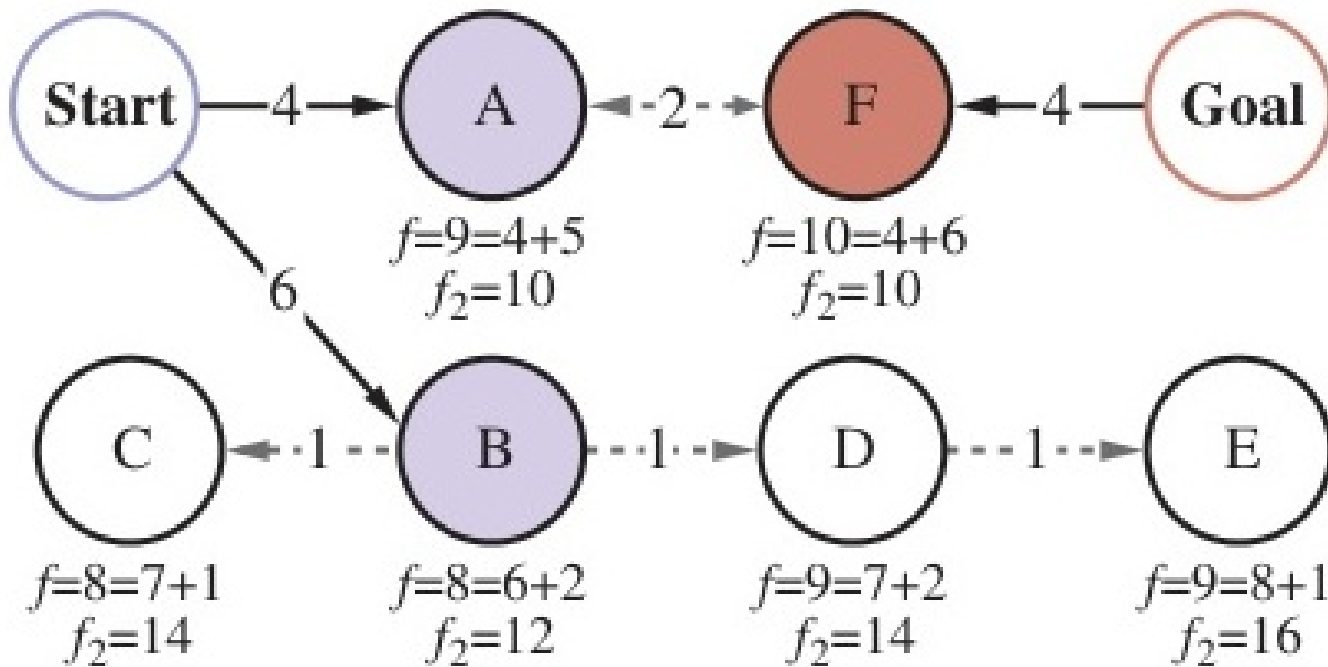


Figure 24: Đồ thị tìm kiếm hai chiều: Mở rộng đồng thời từ điểm xuất phát và mục tiêu.

Ví dụ: Heuristic cho bài toán 8-Puzzle

◇ Xét 2 hàm Heuristic cho bài toán trượt ngói:

- h_1 : Số lượng ô nằm sai vị trí hiện tại so với đích.
- h_2 : Tổng khoảng cách Manhattan của từng ô so với vị trí đích.

◇ Kết quả: Hàm h_2 vượt trội (dominates) hàm h_1 vì $h_2(n) \geq h_1(n)$ với mọi cấu hình. Tốc độ giải A* bằng h_2 sẽ nhanh hơn h_1 hàng nghìn lần!

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Figure 25: Mô phỏng sử dụng Heuristic Manhattan cho bài toán 8-Puzzle.

Ví dụ bài toán 8-Puzzle cụ thể

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Figure 26: Một trường hợp điển hình của 8-puzzle yêu cầu 26 bước giải.

Cơ sở dữ liệu mẫu (Pattern Databases)

*	2	4
*		*
*	3	1

Start State

	1	2
3	4	*
*	*	*

Goal State

Figure 27: Tạo hàm heuristic bằng cách giải quyết một bài toán con của 8-puzzle.

Tính toán trước (Precomputation)

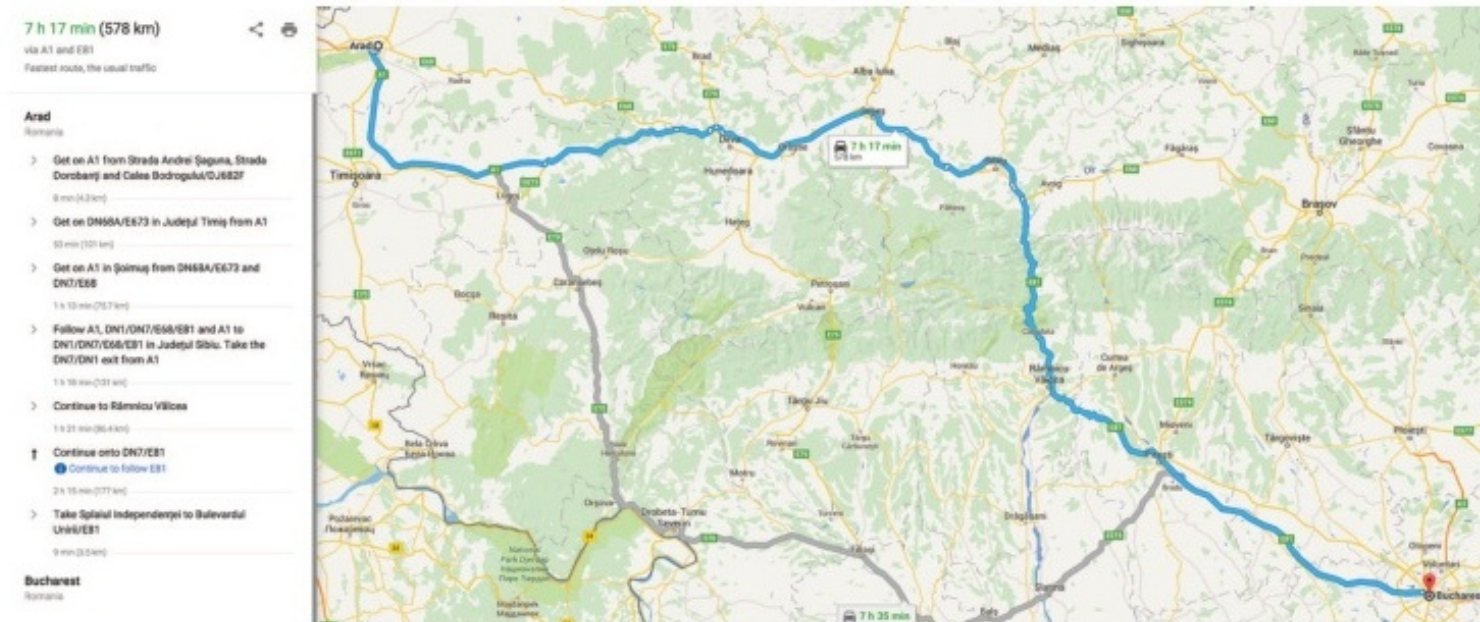


Figure 28: Một hệ thống dịch vụ web tính toán và cung cấp chỉ đường lái xe.

Tóm tắt Chương 3

- ◇ Các tác nhân giải quyết vấn đề mô hình hóa thông qua Không gian Trạng thái, Tập hợp Hành động và Mục tiêu kết thúc.
- ◇ **Tìm kiếm Không thông tin:** BFS an toàn nhưng tốn RAM khủng khiếp. DFS ít tốn RAM nhưng dễ sa lầy. IDS là thuật toán mặc định hoàn hảo kết hợp ưu điểm của BFS và DFS.
- ◇ **Tìm kiếm Có thông tin:** A^* cực kì hiệu quả nhờ sử dụng hàm đánh giá $f(n) = g(n) + h(n)$ để hội tụ nhanh về đích mà vẫn cam kết tìm ra đường ngắn nhất.
- ◇ Khả năng thực chiến và thời gian chạy của A^* phụ thuộc 100% vào việc thiết kế được một **hàm Heuristic Admissible và Consistent**.